# Table of Contents



# List of Figures



# List of Tables

## Introduction

The goal of this project was to design, train, and evaluate convolutional neural network (CNN) models capable of classifying defects on aircraft components through images. These defects include cracks, missing heads, and paint-off. The given dataset has been split into three parts: training, validation, and testing. The Training, Validation, and Test data contain 1942, 431, 538 images, respectively. The image shape of the data was defined to be (500,500,3) in the project outline; however, on further inspection, it was found that the data was in grayscale. Therefore, the image shape was changed to (500, 500, 1) to reflect the true format of the data. Two different models were designed in this project, with each having a unique CNN architecture that produces different results. The first model served as a testing ground for experimentation and learning, while the second model was tuned and optimized, achieving higher accuracy and lower loss.

*Note: The saved models uploaded to GitHub are for a reduced image shape of (100,100). This is because the trained models with an image shape of (500,500) were too large to upload to GitHub. However, all the results in this report are from the model that was trained with an image shape of (500,500).

# Model Evaluation

## Model #1

**Table 1:** Model 1 CNN Architecture

| Layers |
|---|
| Conv2D (32 filters, 3×3, ReLU) |
| MaxPooling (2×2) |
| Conv2D (64 filters, 3×3, ReLU) |
| MaxPooling (2×2) |
| Flatten() |
| Dense (128, ReLU) |
| Dropout (0.5) |
| Dense (3, softmax) |

**Design Rationale:**

1. Convolutional (32→64)
    a. In the first convolutional layer, the model starts with 32 filters to detect simple patterns. In the second convolutional layer, this number is doubled to 64, which allows the network to capture more detail and learn more features.
2. Kernel Size
    a. A kernel size of 3x3 was chosen as it is a standard in modern CNNs because it provides a good field capable of capturing important spatial features. A larger 5x5 size was avoided to prevent overparameterization and overfitting in the case where further layers were added.
3. MaxPooling (2x2)
    a. MaxPooling was used after each convolutional layer to reduce the spatial size of the feature maps. This helps to reduce computation and overfitting while retaining important features.
4. Flatten()
    a. The flattening layer was used to convert the 2-D feature maps into a 1-D vector.
5. Dense Layer (128 neurons)
    a. This layer connects all neurons to previous outputs to combine features and generate final predictions. 128 neurons were initially chosen due to it being a safe medium-sized number. Too few neurons could miss subtle details, while too many

may lead to overfitting. In this first model, 128 seemed like a good baseline for experimentation.

6. Dropout (0.5)
    a. A dropout percentage of 50% was chosen as it is a common standard. Having this dropout layer prevents the model from overfitting by randomly deactivating half the neurons during training.
7. Activation Function
    a. For this case, ReLU was used as it is a common activation function that performs well in most CNNs. It adds nonlinearity to the model, which allows it to learn complex patterns. Relu was preferred over Elu due to its simplicity and lower computational time.
    b. In the final dense layer, the softmax activation function was used because it scales the model's probabilities to 1. This is ideal in multi-classification because it means that all the probabilities for each class add up to 1, which is better for readability and analysis.
8. Epoch
    a. The number of epochs was experimented with quite a bit. It was found that 10 epochs had some underfitting, and 50 epochs resulted in overfitting. An example of overfitting can be seen in Figure 6 of the appendix, where 50 epochs were used. When analyzing Figure 6, it can be seen that the overfitting started to occur after 20 epochs. Due to this, 20 Epochs were chosen to be used.
    b. *Note: Evidence of overfitting was seen in Figure 6 in both graphs, where the validation accuracy and validation loss curves started to plateau away from the training curves. This can be seen by a vertical red line in Figure 6, which shows an estimate of where the model started to overfit.

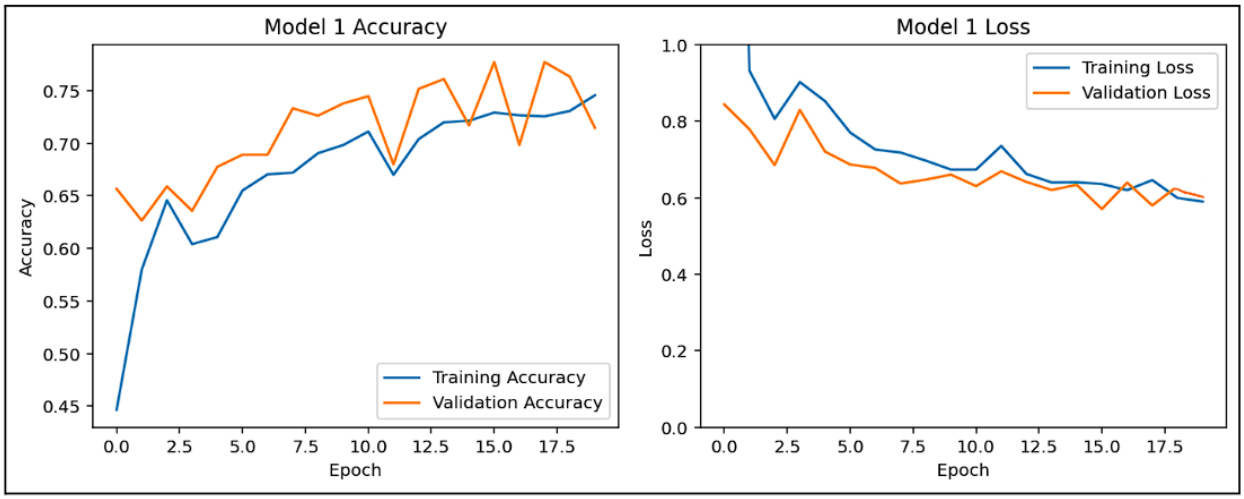


**Figure 1:** Model 1 Performance


**Figure 2:** Model 1 Final Accuracy and Loss

**Outcome:**

In Figure 1, the accuracy and loss graphs for Model 1 showed improvement over the 20 epochs. The loss curve was as expected, with a downward trend for both training and validation, showing that the model is learning effectively. In the accuracy plot, the training accuracy increases from 45% to 75% while the validation accuracy follows similarly, with a peak of around 77%. However, the validation accuracy curve started to fluctuate after 10 epochs, which may indicate the model is learning features but not consistently. There doesn't seem to be any sign of overfitting since neither validation curve is plateauing.

Model 1 achieved a final test accuracy and test loss of 71.06% and 0.7704, respectively. This shows that the model is fairly accurate and can identify defect types; however, there is still room for improvement. The model loss is still quite high, meaning that the model's predictions are far from the target values. This may indicate that the model is not complex enough to learn underlying patterns and fine details. Adding more layers, neurons, and filters may help with this. All of these issues mentioned here will be addressed in Model 2.

## Model #2

**Table 2:** Model 2 CNN Architecture

| Layers |
|---|
| Conv2D (16 filters, 3×3, ReLU) |
| MaxPooling (2×2) |
| Conv2D (32 filters, 3×3, ReLU) |
| MaxPooling (2×2) |
| Conv2D (64 filters, 3×3, ReLU) |
| MaxPooling (2×2) |
| Conv2D (128 filters, 3×3, ReLU) |
| MaxPooling (2×2) |
| Flatten() |
| Dense (256, ReLU) |
| Dropout (0.5) |
| Dense (3, softmax) |

**Design Changes:**

1. Four Convolutional layers
    a. Through experimentation with Model 1 and by analyzing Figure 1, it was seen that the model was not overfitting. This meant the architecture was not overly complex for the dataset. Because of this, it was plausible to add more convolutional layers and filters to increase accuracy and reduce loss.
    b. Model 2 has 4 convolutional layers ranging from 16 filters to 128 filters. Adding more layers allows the model to analyze the images in greater depth and learn more features.
    c. The model first starts with 16 filters, which is fewer than the 32 that model 1 starts with. This allows for a better understanding of global features (big picture).
    d. With each layer, the filters double, which allows the model to capture finer details each time the filter doubles. The last layer has 128 filters, which is more than the 64 that model 1 had. This means model 2 is capable of seeing finer-grained and more complex details.

2. Larger Dense Layer (256 neurons)
    a. Model 1 had 128 neurons in the dense layer; however, this was increased to 256 in Model 2.
    b. The reason for this is that since the convolutional layers are now extracting more detailed features, the dense layer needs to be larger to better combine and interpret all the information. This ensures clear defect type boundaries.

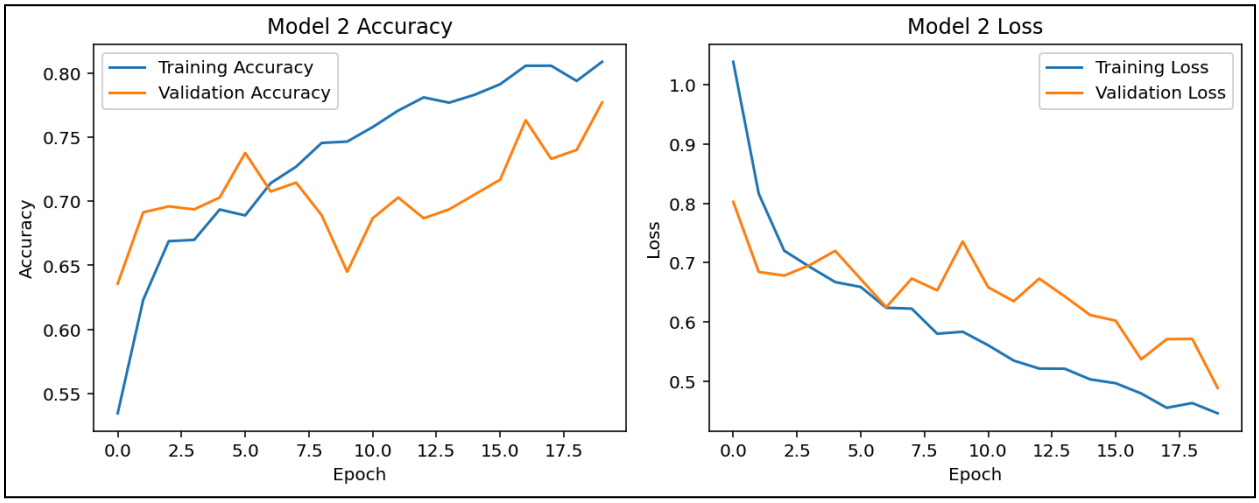


**Figure 3:** Model 2 Performance

```
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.8200 - loss: 0.4978

Model 2 Final Test Accuracy: 0.8200
Model 2 Final Test Loss: 0.4978
```

**Figure 4:** Model 2 Final Accuracy and Loss

**Outcome:**

Looking at the plots in Figure 3, the curve shapes seem to be similar to the ones for Model 1, which is expected; however, this one shows better accuracy and lower loss. Also, this graph does not have the inconsistent, rapidly fluctuating validation accuracy curve from model 1. All of this suggests the design changes made were effective. Furthermore, there is no evidence of overfitting as the validation accuracy is still increasing and the validation loss is still decreasing.

Comparing Figures 4 and 2, it can be seen that Model 2 has made significant improvements, as its accuracy and loss are 82% and 0.4978, respectively. This is a big leap from the previous performance results of Model 1. This shows that the design changes made to the model and the rationale behind them were correct, allowing the network to learn much better. Most of the improvement can be seen in the loss, as it decreased by almost 0.3. This shows that the model is more certain in its predictions and has higher classification confidence. Overall, Model 2 shows significant improvement; its much deeper layered structure allows for better performance, making it a more effective version of Model 1.

## Model Testing

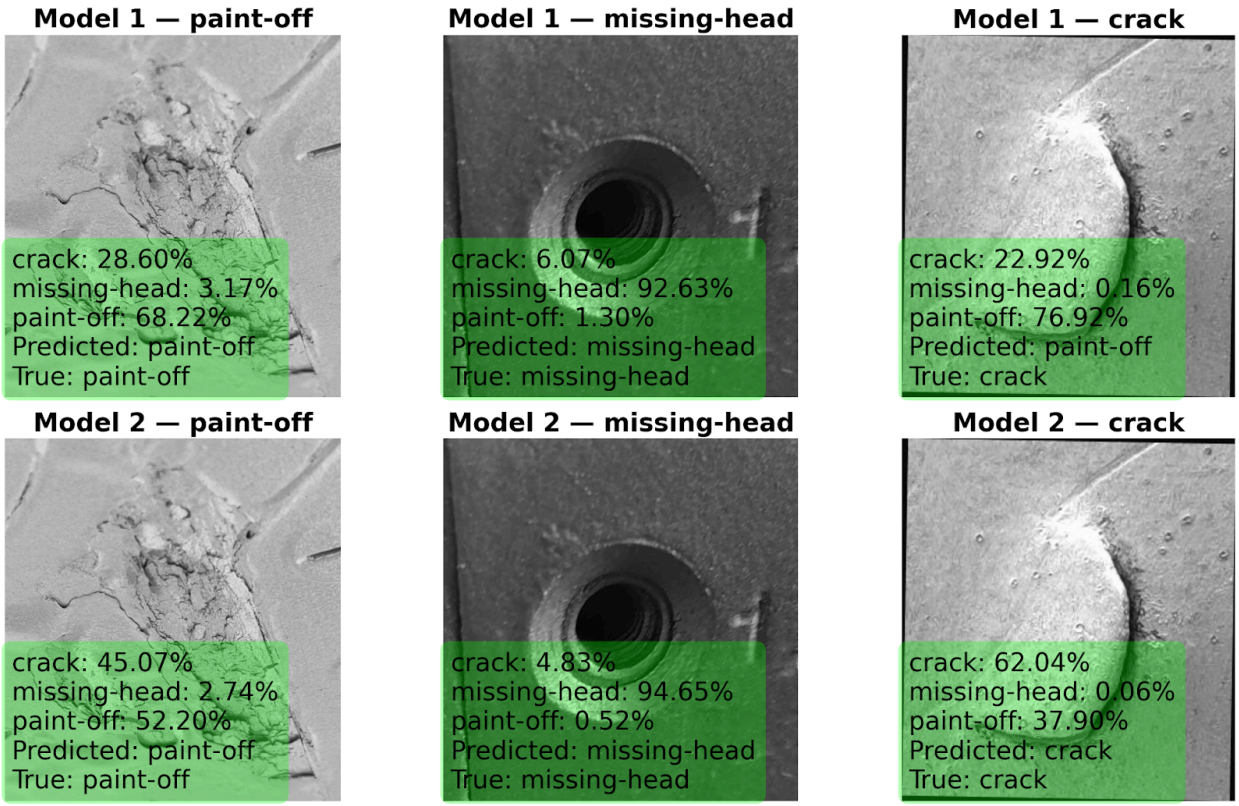


**Figure 5:** Model 1 and Model 2 Testing

Both models were tested on unseen images from the test data. One defect from each category was selected, these being “paint-off”, “missing-head”, and “crack”. From Figure 5, it can be seen that both models correctly identified the “paint-off” and “missing-head” images. However, only Model 2 was able to correctly identify the “crack” image. In the “crack” image, Model 1 gave “paint-off” an incorrect probability of 76.92%. This proves that Model 1 struggles to differentiate between finer details and irregular patterns. The added convolutional layers and higher filters added to Model 2 gave it the capability to capture more detailed spatial information, which allowed it to correctly classify the “crack” image. Overall, Model 2 demonstrates itself to be the superior model as it not only achieved a higher accuracy but also a lower loss. Furthermore, it also produced more confident and correct predications compared to Model 1.

# Appendix

## Model #1 (50 Epoch Variation)

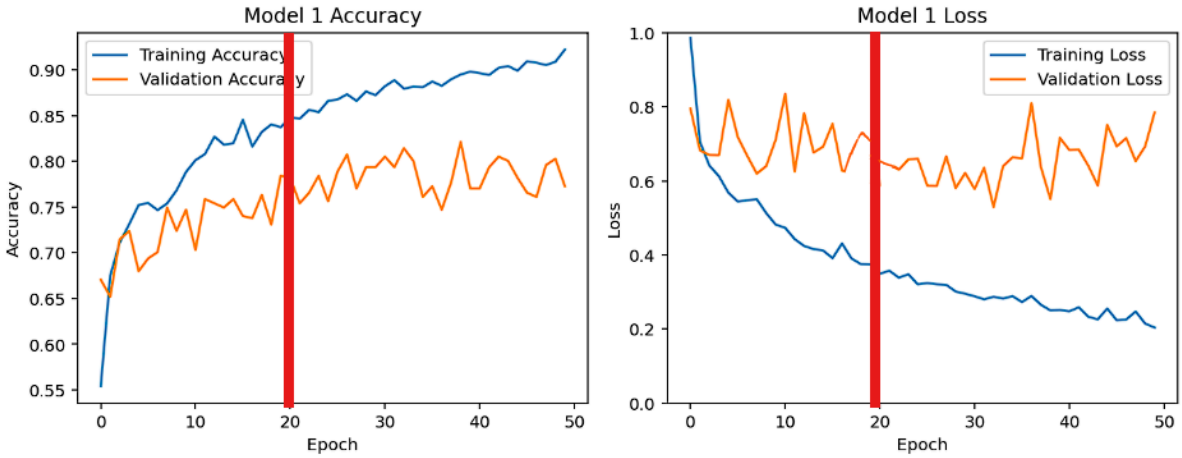


**Figure 6:** Model 1 using 50 Epochs